

Reviewer Pack — Minimal Diophantine Degree Correlation with Communication Co...

Entry 6119353fe426 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 09:59:27 UTC

Minimal Diophantine Degree Correlation with Communication Complexity Growth via LLL Reduction

Entry ID: 6119353fe426 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 09:59:27 UTC

1. Conjecture

Field A (mathematical branch): Diophantine Analysis **Field B** (complexity object): Communication Complexity

Statement:

For every planar graph G , the minimal diophantine degree ($dd(G)$) of its communication complexity is linearly correlated with its communication complexity growth rate ($ccr(G)$), such that $dd(G) = \Theta(ccr(G))$.

Rationale (proposer's reasoning):

Diophantine analysis provides a way to study polynomial equations and their solutions, which can be used to analyze the structure of graphs. The LLL reduction is a well-studied algorithm in diophantine analysis that may reveal underlying structures related to communication complexity. A strong correlation between these two measures could suggest a deep connection between algebraic and combinatorial problems.

Taxonomy category: diophantine_analysis_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e95aa8204ca3bb38

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the computed correlation coefficient (r) between minimal diophantine degree ($dd(G)$) and communication complexity growth rate ($ccr(G)$) for at least 80% of the 30 random seeds is greater than 0.8, with an absolute value of $r \leq 1$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Diophantine Analysis" AND "Communication Complexity" AND LLL reduction" - "minimal diophantine degree" AND communication complexity growth rate" - "0 correlation between diophantine degree and communication complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_planar_graph(n):
        if n == 1:
            return [(0, 1)]
        nodes = list(range(n))
        edges = []
        for i in range(n - 1):
            edges.append((nodes[i], nodes[i + 1]))
        edges.append((nodes[0], nodes[-1]))
        return edges

    def lll_reduction(matrix):
        n = len(matrix)
        B = [list(row) for row in matrix]
        d = [Fraction(0, 1)] * n
        u = [Fraction(1, 1)] * n
        beta = [[Fraction(0, 1)] * n for _ in range(n)]

        def gram_schmidt():
            for k in range(n):
                B[k] = [B[k][i] - sum(beta[k][j] * B[j][i] for j in range(k)) for i in range(len(B[k]))]
                d[k] = Fraction(sum(B[k][i]**2 for i in range(k + 1)), d[k])
                if k > 0:
                    beta[k][k - 1] = Fraction(u[k], d[k - 1])

        def size_reduction():
            for k in range(n):
                for j in range(k - 1, -1, -1):
```

```

        q = B[k][j] // d[j]
        if abs(q) > 0:
            B[k] = [B[k][i] - q * B[j][i] for i in range(len(B[0]))]
            u[k] -= q * u[j]
            beta[k][j] -= q * beta[j][j]
        if k > 1 and abs(u[k]) >= d[k - 1]:
            u[k], u[k - 1] = u[k - 1], u[k]
            B[k], B[k - 1] = B[k - 1], B[k]
        for j in range(k):
            beta[k][j], beta[k - 1][j] = beta[k - 1][j], beta[k][j]

    gram_schmidt()
    size_reduction()

    return [list(row) for row in B]

def minimal_diophantine_degree(graph):
    n = len(graph)
    matrix = [[0] * (n + 2) for _ in range(n + 2)]
    for u, v in graph:
        matrix[u][v] = 1
        matrix[v][u] = 1
        matrix[n][u] += 1
        matrix[n][v] += 1
        matrix[n + 1][n - u] += 1
        matrix[n + 1][n - v] += 1

    reduced_matrix = lll_reduction(matrix)
    return sum(abs(reduced_matrix[i][i]) for i in range(n))

def communication_complexity_rank(graph):
    n = len(graph)
    rank = 0
    for u, v in graph:
        if matrix[u][v] == 1:
            rank += 1
    return rank

def correlation_coefficient(x, y):
    mean_x = sum(x) / len(x)
    mean_y = sum(y) / len(y)
    cov = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(len(x))) / len(x)
    std_x = (sum((x[i] - mean_x)**2 for i in range(len(x))) / len(x))**0.5
    std_y = (sum((y[i] - mean_y)**2 for i in range(len(y))) / len(y))**0.5
    return cov / (std_x * std_y)

n_values = [5, 10, 15, 20, 30, 40]
dd_values = []
ccr_values = []

for n in n_values:
    graph = generate_planar_graph(n)
    dd_G = minimal_diophantine_degree(graph)
    ccr_G = communication_complexity_rank(graph)
    dd_values.append(dd_G)
    ccr_values.append(ccr_G)

```

```

r = correlation_coefficient(dd_values, ccr_values)
return {
    "metric_name": "correlation_coefficient",
    "metric_value": float(r),
    "instances_tested": len(n_values),
    "n_max": max(n_values),
    "conjecture_holds": abs(r) > 0.8 and r >= 0,
    "counterexample": "" if abs(r) > 0.8 and r >= 0 else f"r={r}"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_r = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if abs(result["metric_value"]) > 0.8 and result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_r} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not (abs(result["metric_value"]) > 0.8 and result["conjecture_holds"]))
        print(f"RESULT: FALSIFIED counterexample='r<{mean_r}' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4484acc.py", line 120, in <module>
    trial_result = run_trial(seed)
                   ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4484acc.py", line 100, in run_trial
    dd_G = minimal_diophantine_degree(graph)
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4484acc.py", line 75, in minimal_diophantine_degree
    reduced_matrix = lll_reduction(matrix)
                    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4484acc.py", line 59, in lll_reduction
    gram_schmidt()
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4484acc.py", line 41, in gram_schmidt
    d[k] = Fraction(sum(B[k][i]**2 for i in range(k + 1)), d[k])
               ~~~~~
  File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(0, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete its execution to compute the correlation coefficient. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14119
2	propose	ollama_remote	glm4:latest	0	0	12773
3	preregistration	ollama_remote	glm4:latest	0	0	12366
4	novelty	ollama_remote	glm4:latest	0	0	8547
5	novelty	ollama_remote	glm4:latest	0	0	13162
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21514
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21590
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27862
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21882
10	judge	ollama_remote	glm4:latest	0	0	11954

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 165768 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/6119353fe426.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/6119353fe426.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/6119353fe426.tar.gz (if generated)